Proyecto Oráculo

Selección de configuraciones de prompt con presupuesto de consultas

Sebastián López Dylber Cabrera

Análisis y Diseño de Algoritmos, 2026-2

Grupo y subproyecto	ADAG1, Proyecto Oráculo
Integrantes	Sebastián López y Dylber Cabrera
Repositorio	`https://github.com/Seloga26/ADA2026_2`
Versión entregada	rama `main`, etiqueta `primera-entrega`
Archivos de ADAG1	`entrega.json`, `notebooks/busqueda.ipynb`, `notebooks/evaluacion.ipynb`
Manual técnico	`docs/manual_tecnico.md`

Las decisiones del proyecto se citan por su identificador (D1, D2, ...); el anexo las resume.

Índice

1. Formalización **3**
1.1. Notación común . . . 3
1.2. Parte 1: configuración óptima con oráculo exacto . . . 3
1.3. Parte 2: búsqueda con consultas limitadas o ruidosas . . . 3
1.4. Planteamiento específico de esta entrega . . . 4

2. Reducción desde 3-SAT y análisis de muestreo **4**
2.1. Parte 1: configuración óptima con oráculo exacto . . . 5
2.2. Parte 2: consultas ruidosas o limitadas . . . 5

3. Algoritmo exacto **6**
3.1. Descripción . . . 6
3.2. Correctitud . . . 6
3.3. Costo $T(n)$ . . . 7
3.4. Medición con el modelo sintético . . . 8
3.5. Alcance con el modelo real . . . 9

4. Algoritmos aproximados **9**
4.1. Descripción del algoritmo . . . 9
4.2. Propiedades . . . 10
4.3. Costo $T(n)$ . . . 10
4.4. Cota de calidad: no hay razón de aproximación . . . 11
4.5. Qué no se afirma . . . 11

5. Comparación de reparto del presupuesto **12**

6. Proceso de optimización y resultados **12**
6.1. Método, parámetros y parada 12
6.2. Registro de las evaluaciones 13
6.3. Trazas de fallos y decisiones derivadas 14
6.4. Validación final 15
6.5. Límites 15
6.6. Dónde está cada cosa 16

7. Discrepancias y limitaciones **16**
7.1. Discrepancias entre las fuentes 16
7.2. Limitaciones de los resultados 17

Anexo: decisiones citadas **18**

1. Formalización

El problema se formaliza en dos partes, cada una con el mismo esquema: contexto, entradas y tamaño, salidas, función objetivo y restricciones, y supuestos y alcance. La demostración de NP-dureza y el análisis de muestreo están en la sección 2; aquí solo se fija el problema que ambas estudian.

1.1. Notación común

Definición 1 (Espacio de configuraciones)**.** Hay L categorías y la categoría i ofrece k_i opciones. Una configuración elige una opción por categoría. El conjunto de configuraciones es $\mathcal{C}$ y $|\mathcal{C}| = \prod_{i=1}^{L} k_i$. La temperatura de generación también forma parte de la configuración; en este proyecto se fija en 0,0, de modo que no multiplica el espacio (con las tres temperaturas disponibles serían $3 \cdot 32\,768 = 98\,304$ configuraciones). Un lote I de $n = |I|$ instancias fija las pruebas: cada instancia exige un conjunto de restricciones sobre la respuesta. Una configuración *cumple* una instancia si satisface todas sus restricciones; si falta alguna, no la cumple (puntaje todo-o-nada). Con eso, $\mathrm{Acc}(c)$ es la fracción de instancias de I que c cumple por completo.

Además, B es el presupuesto de generaciones y m_j las consultas gastadas en la configuración c_j. Las dos partes difieren en quién decide si una configuración cumple una instancia. En la parte 1 lo decide un modelo de reglas explícito, que forma parte de la entrada y es el mismo de CONFIG-EXACTA (sección 2). En la parte 2 lo decide la respuesta que genera el modelo de lenguaje, revisada por los verificadores del oráculo.

1.2. Parte 1: configuración óptima con oráculo exacto

Contexto. Se supone un oráculo que entrega $\mathrm{Acc}(c)$ exacto, sin ruido y sin límite de consultas. Es una abstracción para estudiar la dureza del problema; el problema real de esta entrega es la parte 2.

Entrada y tamaño. La entrada es $(L, k_1, \ldots, k_L, R, \sigma, I)$: el catálogo, un conjunto R de reglas, la función σ que indica qué reglas de R satisface cada opción y el lote I, donde cada instancia exige un subconjunto de R. Una configuración cumple una instancia si la unión de las reglas de sus opciones contiene todas las reglas exigidas. Su tamaño no se reduce a una sola variable: depende de L, de los k_i, de $|R|$ y de n. Como en la sección 2, L y los k_i forman parte de la entrada, de modo que $|\mathcal{C}|$ puede crecer de forma exponencial en L. El proyecto es el caso $L = 5$ y $k_i = 8$, donde $|\mathcal{C}| = 8^5 = 32\,768$ es una constante: con L fijo el espacio no crece, aunque siga siendo grande para recorrerlo con un modelo real.

Salida. Una configuración $c \in \mathcal{C}$.

Objetivo y restricciones. Maximizar $\mathrm{Acc}(c)$ sobre $c \in \mathcal{C}$. No hay presupuesto de consultas.

Supuestos y alcance. Con σ en la entrada, el oráculo exacto calcula $\mathrm{Acc}(c)$ en tiempo polinomial. Su versión de decisión, con un umbral t, es CONFIG-EXACTA, cuya NP-completitud se demuestra en la sección 2. Esta parte solo sirve para el análisis de dureza: el modelo de lenguaje real no se describe con un σ conocido.

1.3. Parte 2: búsqueda con consultas limitadas o ruidosas

Contexto. El oráculo es un modelo de lenguaje y cada consulta es una generación en GPU. Con temperatura 0,0 la generación es, en principio, determinista (D2). En GPU pueden aparecer diferencias numéricas pequeñas; esto es una inferencia, no una medición. El caché hace

repetibles las consultas ya hechas. Por eso la variabilidad relevante no viene del modelo sino de qué instancias se miden, en coherencia con el análisis de la sección 2.

Entrada y tamaño. El mismo espacio $(L, k_1, \ldots, k_L)$, el lote de instancias I, el presupuesto B y la semilla. El tamaño vuelve a depender de L, de los k_i y de n.

Contabilidad de B. B cuenta las generaciones nuevas por (configuración, instancia, semilla) (D11). Una terna ya generada y guardada en el caché no se cobra.

Salida. Las consultas m_j por configuración c_j y una configuración ganadora c^*.

Objetivo y restricciones. Maximizar $\mathrm{Acc}(c^*)$ sujeto a $\sum_j m_j \leq B$, con m_j entero y $0 \leq m_j \leq n$: como el caché no cobra lo ya generado, una configuración no consume más de n generaciones nuevas sobre el lote. Cuando $m_j < n$, $\mathrm{Acc}(c_j)$ no se observa; solo se conoce la estimación $\widehat{\mathrm{Acc}}_{m_j}(c_j)$, la fracción de aciertos en las m_j instancias medidas. Por eso c^* se elige con las estimaciones disponibles y la calidad de la elección depende de cuántas consultas se gastaron en cada configuración; este es el problema que la sección 2 analiza por complejidad de muestreo.

Supuestos y alcance. c^* es la mejor configuración encontrada, no un óptimo demostrado: el algoritmo no recorre $\mathcal{C}$ ni garantiza haberlo hecho. Lo que $\mathrm{Acc}(c^*)$ mide es el desempeño sobre el lote de búsqueda, no fuera de él.

1.4. Planteamiento específico de esta entrega

Elemento	Valor
Espacio explorado	5 ranuras de 8 opciones, $\lvert\mathcal{C}\rvert = 32\,768$
Temperatura	fija en 0,0
Lote de búsqueda	100 instancias de `datos_visibles.json`
Métrica	Acc, todo-o-nada
Modelo y semilla	`qwen17b`, semilla 1
Presupuesto	$B = 14\,400$ generaciones (D17)
Validación	87 instancias, una sola vez sobre la configuración elegida

La validación se usa solo después de elegir la configuración: si interviniera en la elección, dejaría de ser una prueba independiente.

Fuera de esta entrega. El backtracking se implementó y se midió solo con el modelo sintético (sección 3). Quedan fuera el reparto uniforme frente a UCB1 y el exacto sobre un subcatálogo con el modelo real (D17). Successive Halving se descartó y no se implementa (D4).

2. Reducción desde 3-SAT y análisis de muestreo

Esta sección separa las dos partes del problema. La primera supone un oráculo exacto y justifica su NP-dureza. La segunda considera consultas ruidosas o limitadas y se analiza por complejidad de muestreo: no hereda la demostración de la primera.

2.1. Parte 1: configuración óptima con oráculo exacto

La dureza se estudia sobre el problema general, donde el número de categorías L y las opciones $k_1, \ldots, k_L$ forman parte de la entrada. Con $L = 5$ fijo, $|\mathcal{C}|$ sería constante y bastaría recorrer todas las configuraciones. Por eso el caso del proyecto, con $L = 5$ y $k_i = 8$, es una instancia finita del problema general y no hereda por sí sola la dureza.

Definición 2 (CONFIG-EXACTA)**.** Se da un catálogo de L categorías con k_i opciones, un conjunto de reglas, las reglas que satisface cada opción, n instancias, cada una con las reglas que exige, y un umbral $t \in [0, 1]$. Una configuración c elige una opción por categoría y cumple una instancia si la unión de las reglas de sus opciones contiene todas las reglas exigidas. Con $\mathrm{Acc}(c)$ la fracción de instancias cumplidas, se pregunta si existe c con $\mathrm{Acc}(c) \geq t$.

El oráculo exacto calcula $\mathrm{Acc}(c)$ en tiempo polinomial en el tamaño del catálogo y de las instancias. La entrada es el catálogo y no la lista de configuraciones, que tiene $\prod_i k_i$ elementos.

Teorema 1. *CONFIG-EXACTA es NP-completo, y maximizar* $\mathrm{Acc}(c)$ *es NP-difícil.*

Demostración. Pertenencia a NP. El certificado es una configuración c, de tamaño polinomial. Calcular $\mathrm{Acc}(c)$ y compararla con t toma tiempo polinomial.

Construcción. Sea φ una fórmula 3-SAT, problema NP-completo [3], con variables $x_1, \ldots, x_m$ y cláusulas $D_1, \ldots, D_q$. Se construye:

1. $L = m$ categorías, una por variable, con dos opciones: *falso* y *verdadero*.
2. Una regla r_j por cláusula.
3. La opción *verdadero* de x_i satisface r_j si x_i aparece en D_j; la opción *falso* la satisface si $\neg x_i$ aparece en D_j.
4. $n = q$ instancias, donde la instancia j exige solo r_j.
5. El umbral $t = 1$.

El catálogo tiene $2m$ opciones y hay q reglas e instancias, por lo que la construcción toma tiempo $O(m + q)$. Aunque $|\mathcal{C}| = 2^m$, las configuraciones no se escriben en la entrada.

Las configuraciones corresponden una a una con las asignaciones de verdad. Para una configuración c y su asignación α, la instancia j se cumple si y solo si alguna opción elegida satisface r_j, es decir, si y solo si α hace verdadero un literal de D_j.

(⇒) Si α satisface φ, cada cláusula tiene un literal verdadero. Entonces c cumple todas las instancias y $\mathrm{Acc}(c) = 1 \geq t$.

(⇐) Si existe c con $\mathrm{Acc}(c) \geq 1$, cumple las q instancias. Por la equivalencia anterior, su asignación hace verdadera cada cláusula y satisface φ.

Por lo tanto, φ es satisfacible si y solo si la instancia construida tiene respuesta afirmativa, y CONFIG-EXACTA es NP-completo. Un algoritmo polinomial para la optimización decidiría el problema comparando el máximo con t, así que maximizar $\mathrm{Acc}(c)$ es NP-difícil. □

Ejemplo. Para $(x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3)$, la asignación $x_1 =$ verdadero, $x_2 =$ falso, $x_3 =$ falso satisface ambas cláusulas. La configuración que elige esas opciones cumple las dos instancias y tiene $\mathrm{Acc}(c) = 1$.

2.2. Parte 2: consultas ruidosas o limitadas

En la práctica el oráculo es un modelo de lenguaje, cada consulta es una generación en GPU y el puntaje se mide sobre instancias finitas. Ya no se pregunta solo qué configuración es óptima,

sino cuántas consultas hacen falta para distinguirla. La demostración anterior no se traslada a este caso.

Supóngase que las consultas de una configuración c son variables de Bernoulli independientes con media $\mathrm{Acc}(c)$, y sea $\widehat{\mathrm{Acc}}_m(c)$ la media de m consultas. La desigualdad de Hoeffding [2] da

$$\Pr\big[\,|\widehat{\mathrm{Acc}}_m(c) - \mathrm{Acc}(c)| \geq \varepsilon\,\big] \leq 2e^{-2m\varepsilon^2}.$$

Con la cota de la unión sobre $\mathcal{C}$, todas las estimaciones tienen error menor que ε con probabilidad al menos $1-\delta$ si

$$m \geq \frac{\ln(2|\mathcal{C}|/\delta)}{2\varepsilon^2}.$$

Separar dos configuraciones con diferencia Δ exige una cantidad de consultas del orden de $\ln(1/\delta)/\Delta^2$.

Aplicación. Con $|\mathcal{C}| = 32\,768$, $\varepsilon = 0{,}05$ y $\delta = 0{,}05$ se obtiene $m \geq 2\,818$ consultas por configuración, unos 92,3 millones en total. Esto supera las 100 instancias de búsqueda y muestra que una garantía uniforme sobre todo el espacio es inviable: la búsqueda debe concentrar el presupuesto.

Dos límites completan el análisis. Sobre el lote fijo de 100 instancias, con temperatura 0, medir todas da la precisión exacta en ese lote, con resolución de un punto porcentual, pero no garantiza nada fuera de él. Por otro lado, para una configuración fijada antes de mirar los datos, 87 instancias de validación dan un error de hasta 0,146 con $\delta = 0{,}05$. Por eso la validación se usa una sola vez y su resultado se reporta como estimación.

3. Algoritmo exacto

El exacto es un backtracking con cota (D3, D5). Está en el módulo `backtracking.py` de `busqueda_prompt/algoritmos`, se comprobó contra la fuerza bruta y se midió con el modelo sintético (D7). Con el modelo real no se ejecutó: esta sección no reporta ninguna precisión real ni afirma un óptimo global o de un subcatálogo.

3.1. Descripción

El árbol de búsqueda tiene un nivel por ranura: un nodo de nivel ℓ es una asignación parcial de las ranuras $1, \ldots, \ell$ y sus hijos prueban las $k_{\ell+1}$ opciones de la siguiente, en orden creciente. Las hojas son las configuraciones de $\mathcal{C}$, recorridas en orden lexicográfico, el mismo de la fuerza bruta. En cada hoja la configuración se evalúa instancia por instancia, y se lleva la mejor cantidad de aciertos conocida, a^*.

La cota D5 no supone nada del oráculo: una instancia aún no evaluada aporta a lo sumo un acierto, así que $a + (n - j + 1)$ acota los aciertos finales de la configuración. Vale con el modelo real y con el sintético, con o sin ruido.

La cota parcial U solo se usa con el modelo sintético, donde la estructura de reglas es conocida (D6). Para una asignación parcial p, las reglas alcanzables son las de las opciones elegidas más todas las de las ranuras libres; $U(p)$ cuenta las instancias cuyas reglas exigidas están todas en ese conjunto. Ninguna configuración que complete p cubre una regla fuera de él, y los conflictos solo quitan reglas, así que $U(p)$ es una cota superior. Con ruido deja de valer y el código la rechaza. Con el modelo real no hay un σ conocido y no se poda con supuestos.

3.2. Correctitud

Teorema 2 (Correctitud)**.** *Si U es una cota superior válida, o no se usa, el algoritmo 1 devuelve la primera configuración del orden lexicográfico con la mayor precisión, la misma que la fuerza bruta.*

Algorithm 1 Backtracking con cota

Require: ranuras $1, \dots, L$ con k_i opciones, lote $I = (x_1, \dots, x_n)$, cota parcial U opcional

$a^* \leftarrow -1$; $c^* \leftarrow$ ninguna
procedure EXPLORAR(ℓ, p) ▷ p asigna las ranuras $1, \dots, \ell$
  if $\ell = L$ **then**
    $a \leftarrow 0$
    for $j = 1, \dots, n$ **do**
      if $a + (n - j + 1) \le a^*$ **then return** ▷ cota D5: no puede superar a c^*
      end if
      if p cumple x_j **then** $a \leftarrow a + 1$ ▷ una generación
      end if
    end for
    if $a > a^*$ **then** $a^* \leftarrow a$; $c^* \leftarrow p$
    end if
    return
  end if
  if U dada, $\ell > 0$, $a^* \ge 0$ y $U(p) \le a^*$ **then return** ▷ poda por estructura (D6)
  end if
  for $o = 0, \dots, k_{\ell+1} - 1$ **do**
    EXPLORAR($\ell + 1$, $p \cup \{\ell + 1 \mapsto o\}$)
  end for
end procedure
EXPLORAR(0, $\varnothing$); **return** c^*, a^*/n

Demostración. Sea c° la primera configuración del orden con el máximo de aciertos a°. Toda configuración anterior tiene menos de a° aciertos, de modo que al llegar a c° se cumple $a^* < a^\circ$. La cota D5 no la abandona: en cada paso j, sus aciertos finales cumplen $a^\circ \le a + (n - j + 1)$, y por tanto $a + (n - j + 1) > a^*$. Ningún ancestro de c° se poda por U: si p es ancestro, $U(p) \ge a^\circ > a^*$. Entonces c° se evalúa completa y pasa a ser c^*. Después, ninguna configuración tiene más de a° aciertos y el reemplazo exige $a > a^*$, así que c^* ya no cambia. Las configuraciones que se abandonan o se podan no podían superar estrictamente a a^*, así que no se pierde ninguna que la fuerza bruta habría elegido. □

Las pruebas de `tests/test_backtracking.py` comprueban el teorema: en 70 instancias sintéticas, con conflictos, con ruido y con la cota estructural, la ganadora y la precisión coinciden con las de la fuerza bruta. También comprueban, en 10 casos sintéticos, que U nunca queda por debajo de los aciertos reales en ninguna asignación parcial.

3.3. Costo $T(n)$

El costo se mide en generaciones, como en la sección 4: cada vez que se pregunta si p cumple x_j se hace una generación (D11).

Teorema 3 (Costo)**.** *El algoritmo 1 hace a lo sumo $|\mathcal{C}| \cdot n$ generaciones, y esa cota se alcanza. Visita a lo sumo $\sum_{\ell=0}^{L} \prod_{i \le \ell} k_i$ nodos, que es menos de $2|\mathcal{C}|$ si todo $k_i \ge 2$. Sin U, el tiempo total es $O(|\mathcal{C}| \cdot (n + L))$, y el número de generaciones es $\Theta(|\mathcal{C}| \cdot n)$ en el peor caso.*

Demostración. Cada hoja evalúa a lo sumo sus n instancias y hay $|\mathcal{C}|$ hojas. Para el número de nodos, el nivel ℓ tiene $\prod_{i \le \ell} k_i \le |\mathcal{C}|/2^{L-\ell}$ nodos si todo $k_i \ge 2$, y la suma geométrica queda por debajo de $2|\mathcal{C}|$. Un nodo interno cuesta $O(1)$ sin U, y una hoja $O(n + L)$, contando la copia de la configuración, así que el tiempo es $O(|\mathcal{C}| \cdot (n + L))$. La cota se alcanza con un oráculo en el que ninguna configuración cumple ninguna instancia: a^* queda en 0 tras la primera hoja, y en

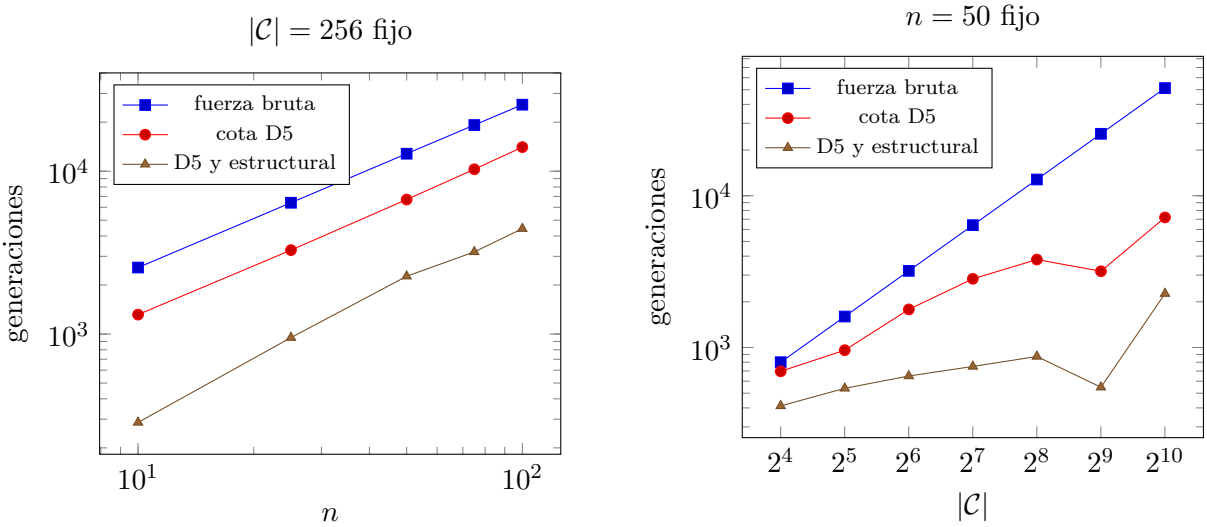
cada hoja la condición de abandono $0 + (n - j + 1) \leq 0$ nunca se cumple, así que todas las hojas evalúan sus n instancias. □

En el mejor caso, si la primera configuración cumple todo, $a^* = n$ y las demás se abandonan antes de su primera generación: el costo es n. Ambos extremos están cubiertos por pruebas automáticas. Con U, cada nodo interno suma $O(n \cdot |R|)$ operaciones sin generaciones.

Con L y los k_i como parte de la entrada, $|\mathcal{C}| = \prod_i k_i \geq 2^L$ crece de forma exponencial, lo cual es coherente con la NP-dureza de la sección 2: la poda mejora casos concretos, pero no cambia el peor caso. En el caso del proyecto, con $|\mathcal{C}|$ fijo en 32 768, el costo es lineal en n con una constante enorme.

3.4. Medición con el modelo sintético

El barrido sigue D10: $|\mathcal{C}| = 256$ fijo (4 ranuras de 4 opciones) con $n \in \{10, 25, 50, 75, 100\}$, y $n = 50$ fijo con $|\mathcal{C}| = 2^L$ para L de 4 a 10. Cada punto promedia 5 semillas de instancias con 12 reglas, 3 reglas por instancia, densidad 0,2 y 3 conflictos. En todas las semillas el exacto coincidió con la fuerza bruta. Se ejecuta con `python -m busqueda_prompt.experimentos.medir`, que escribe `resultados/exacto/barrido_n.csv`, `barrido_c.csv` y `ajuste.json`.



Pendiente log-log de las generaciones	frente a n	frente a $\lvert\mathcal{C}\rvert$
Fuerza bruta y peor caso	1,00	1,00
Backtracking con cota D5	1,03	0,52
Backtracking con D5 y cota estructural	1,19	0,28

Lectura. Frente a n, la cota D5 ahorra una fracción casi constante, entre el 45 y el 49 % con $|\mathcal{C}| = 256$ (14 048 frente a 25 600 generaciones con $n = 100$), y el costo sigue siendo lineal en n, como predice el teorema. Frente a $|\mathcal{C}|$, el ahorro crece: con $|\mathcal{C}| = 1\,024$ la cota D5 gastó 7 195 generaciones frente a 51 200, porque al encontrar pronto una configuración con muchos aciertos casi todas las demás se abandonan tras pocas instancias. La cota estructural poda subárboles enteros y baja más el gasto, aunque su pendiente frente a n es algo mayor porque su ventaja relativa es mayor con n pequeño.

Estas pendientes describen este generador y estos parámetros; no son una garantía. Al crecer L con densidad fija, cada instancia se cubre con más facilidad y la precisión máxima sube, lo que favorece la poda, y la serie frente a $|\mathcal{C}|$ no es monótona (512 gastó menos que 256). El peor caso, que se midió con el oráculo sin aciertos, mantiene la pendiente 1 en los dos ejes. Los tiempos de pared fueron de 0,004 a 0,09 s por ejecución y siguen a las generaciones; con el modelo real dominan las generaciones.

3.5. Alcance con el modelo real

Con 1,99 s por generación medidos en T4, el peor caso del espacio completo serían 3 276 800 generaciones, unos 75 días de GPU. En un subcatálogo con las 3 mejores opciones por ranura (243 configuraciones, D8) el peor caso serían 24 300 generaciones, unas 13 h, y con 2 por ranura (32 configuraciones), 3 200, cerca de 1,8 h. Ninguno se ejecutó en esta entrega, porque el presupuesto $B = 14\,400$ se dedicó a la búsqueda por coordenadas (D17). Un óptimo de subcatálogo solo sería óptimo en ese subcatálogo, y el subcatálogo mismo saldría de la búsqueda, no de una demostración.

4. Algoritmos aproximados

El aproximado principal es la búsqueda por coordenadas (D3). Es la técnica que se implementó y se ejecutó con el modelo real; sus resultados están en la sección 6. El reparto uniforme frente a UCB1 no se desarrolla en esta entrega (sección 5) y aquí no se le atribuye ninguna cota.

4.1. Descripción del algoritmo

Se usa la notación de la sección 1: L ranuras, la ranura i con k_i opciones y $\mathrm{Acc}(c)$ la fracción de instancias del lote I que c cumple por completo. Para cada ranura i hay un conjunto de opciones permitidas $O_i \subseteq \{0, \ldots, k_i - 1\}$, que por defecto es todo el rango. Reducirlo es la forma de aplicar lo que enseñan las trazas de fallos (sección 6). Sea π un orden de las ranuras y P el tope de pasadas. Escribimos $c[i{:=}o]$ para la configuración que sale de c cambiando solo la ranura i a la opción o.

Algorithm 2 Coordenadas con un arranque

Require: orden π, opciones permitidas O_i, tope de pasadas P, presupuesto B

elegir c al azar, con $c_i \in O_i$ para toda ranura i ▷ semilla fija
$a \leftarrow \mathrm{Acc}(c)$
for $t = 1, \ldots, P$ **do**
    mejoró $\leftarrow$ falso
    for cada ranura i en el orden π **do**
        for cada opción $o \in O_i$ con $o \neq c_i$ **do**
            $c' \leftarrow c[i{:=}o]$; $a' \leftarrow \mathrm{Acc}(c')$
            if $a' > a$ **then** ▷ mejora estricta
                $c \leftarrow c'$; $a \leftarrow a'$; *mejoró* $\leftarrow$ verdadero
            end if
        end for
    end for
    if no *mejoró* **then**
        salir ▷ pasada completa sin cambios
    end if
end for
return c

Dos detalles de la implementación (`coordenadas.py`) que el pseudocódigo respeta. La comparación $o \neq c_i$ usa la opción *actual* en ese momento: tras aceptar un cambio dentro de una ranura, la opción anterior vuelve a probarse más adelante en esa misma ranura (sale del caché, sin generaciones nuevas). Y si el presupuesto B se agota, el algoritmo devuelve la mejor configuración vista hasta ahí, que es la actual por el lema 1. Varios arranques repiten el algoritmo desde puntos de partida distintos y se devuelve la mejor de las configuraciones finales. Entre un

arranque y otro pueden cambiar π y los O_i; en ese caso los arranques no son independientes, como ocurrió en este proyecto (sección 6).

4.2. Propiedades

Lema 1 (Monotonía)**.** *La precisión de la configuración actual no decrece, y cada cambio la aumenta de forma estricta. En particular, la configuración devuelta es al menos tan buena como la de partida.*

Demostración. La configuración solo cambia en la línea con la condición $a' > a$, y en ese caso a pasa a a', estrictamente mayor. Si no hay cambio, c y a quedan igual. □

Lema 2 (Terminación)**.** *Con* Acc *determinista, el algoritmo hace a lo sumo n cambios y a lo sumo* $\min(P,\, n+1)$ *pasadas.*

Demostración. $\mathrm{Acc}(c)$ toma valores en $\{0, \frac{1}{n}, \ldots, 1\}$, así que cada cambio lo sube al menos $\frac{1}{n}$, y como parte de al menos 0 y no pasa de 1, solo cabe n veces. Toda pasada que no es la última contiene al menos un cambio, de modo que hay a lo sumo n pasadas con cambios más la pasada final sin cambios. □

Teorema 4 (Óptimo local por ranura)**.** *Si el algoritmo termina porque una pasada completa no cambió nada, la configuración c devuelta cumple* $\mathrm{Acc}(c[i{:=}o]) \le \mathrm{Acc}(c)$ *para toda ranura i y toda opción $o \in O_i$.*

Demostración. En esa pasada la configuración no cambió, así que todos los candidatos $c[i{:=}o]$ se compararon contra el mismo $a = \mathrm{Acc}(c)$ y ninguno lo superó estrictamente. Cada candidato con $o \in O_i$ y $o \neq c_i$ se evaluó en esa pasada; el caso $o = c_i$ da c mismo. □

El teorema depende de que Acc devuelva el mismo valor cada vez que se evalúa una configuración. Con un oráculo exacto es inmediato; con el modelo real se apoya en la temperatura 0,0 y en el caché, que devuelve lo ya generado (D2).

Es un óptimo *por ranura*: ningún cambio de una sola ranura mejora, pero no se prueba ningún cambio simultáneo de dos ranuras. Además, si O_i se reduce a un subconjunto, el óptimo es local respecto a ese subespacio, no respecto a todas las opciones. La sección 6 muestra que esto ocurrió en el arranque 1.

4.3. Costo $T(n)$

El costo se mide en generaciones, que es lo que cobra el presupuesto B (D11): una generación tarda unos 1,99 s en la GPU del proyecto, mucho más que cualquier otra operación del algoritmo. Evaluar una configuración sobre I cuesta n generaciones, una por instancia.

Teorema 5 (Costo)**.** *Sea $p \le \min(P, n+1)$ el número de pasadas realizadas. El número de evaluaciones de configuraciones es a lo sumo $1 + p\sum_i |O_i|$, y las generaciones, a lo sumo*

$$T(n) \;\le\; n\Big(1 + p\sum_{i=1}^{L} |O_i|\Big) \;=\; O(p \cdot L \cdot k_{\max} \cdot n).$$

Demostración. En una pasada, el recorrido de la ranura i examina cada opción de O_i una vez, y evalúa todas menos la actual, así que hace a lo sumo $|O_i|$ evaluaciones. Sumar sobre las ranuras da $\sum_i |O_i|$ por pasada, más la evaluación inicial. Cada evaluación son n generaciones. El caché solo puede bajar la cifra, porque no cobra una terna (configuración, instancia, semilla) ya generada. □

Con P constante, el costo es lineal en n. Sin tope, el lema 2 da $p \leq n+1$ y el peor caso sube a $O(L \cdot k_{\text{máx}} \cdot n^2)$. El costo no depende de $|\mathcal{C}| = \prod_i k_i$, a diferencia del recorrido exhaustivo, que es $O(|\mathcal{C}| \cdot n)$.

Para el caso del proyecto, $L = 5$, $k_i = 8$ y $n = 100$:

Concepto	Generaciones, cota
Una pasada, sin exclusiones: $5 \cdot 8 \cdot 100$	4 000
Un arranque con $P = 3$: $100\,(1 + 3 \cdot 40)$	12 100
Presupuesto B (D17)	14 400

Dos arranques de $P = 3$ podrían sumar, en el peor caso, más que B. Dentro de una ejecución, lo que acota el gasto es el contador de generaciones, que lanza `PresupuestoAgotado` antes de pasar de B. Los dos arranques reales corrieron en sesiones distintas, cada una con su propio contador, así que el total conjunto frente a B se comprobó después, sobre la unión de los historiales. Lo observado fue menor y está en la sección 6: el arranque 0 evaluó 92 configuraciones distintas (9 200 generaciones) y el arranque 1, 28 (2 800).

4.4. Cota de calidad: no hay razón de aproximación

El algoritmo garantiza el lema 1 y el teorema 4, pero no una razón de aproximación respecto al óptimo.

Teorema 6 (Sin razón de aproximación)**.** *No existe una constante $\alpha > 0$ tal que el algoritmo 2 devuelva siempre c con $\text{Acc}(c) \geq \alpha\ \text{Acc}(c^*)$, donde c^* es un óptimo.*

Demostración. Sean $L = 2$ ranuras con dos opciones cada una, y un lote de n instancias idénticas en el que cumplir la instancia exige dos condiciones: una la satisface solo la opción 1 de la ranura 1 y la otra, solo la opción 1 de la ranura 2. Por el puntaje todo-o-nada, $\text{Acc}(1,1) = 1$ y $\text{Acc}(c) = 0$ para las otras tres configuraciones. Si el punto de partida es $(0,0)$, sus dos vecinos tienen precisión 0, no hay mejora estricta y el algoritmo termina en $(0,0)$ con $\text{Acc} = 0$, frente a $\text{Acc}(c^*) = 1$. La cantidad de ranuras y opciones del proyecto solo agrega ranuras irrelevantes y no altera este argumento. □

Este es un ejemplo construido en el modelo, no una observación con el modelo real. Lo que lo hace posible es el puntaje todo-o-nada, que crea mesetas donde ningún vecino mejora. Lo que sí se observó con el modelo real va en la misma dirección: dos arranques con puntos de partida y espacios distintos terminaron en 20 % y 17 % (sección 6), y la mejor opción de estilo depende de la estrategia elegida. Por eso el riesgo de quedar en un óptimo local se mide con varios arranques y no se declara cubierto.

4.5. Qué no se afirma

- No se afirma que la configuración encontrada sea un óptimo global ni del subcatálogo. Es la mejor encontrada.
- No hay garantía probabilística de haber hallado el óptimo: el análisis de muestreo de la sección 2 muestra que una garantía uniforme sobre todo el espacio es inviable con este presupuesto.
- La precisión de búsqueda es la de un lote fijo de 100 instancias y no se extiende sin supuestos a otras instancias.

5. Comparación de reparto del presupuesto

La comparación entre el reparto uniforme y UCB1 [1] no se desarrolla en la primera entrega: ninguno de los dos está implementado y no hay resultados que reportar.

Successive Halving se descartó y no se implementa (D4). La tabla compara su costo sobre $|\mathcal{C}| = 32\,768$ con el de otras estrategias, en generaciones.

Estrategia	Generaciones
Fuerza bruta con $n = 100$	3 276 800
Successive Halving según la fórmula, $n_0 = 2$	983 040
Successive Halving con tope $n = 100$	495 416
Successive Halving con tope y caché	247 808
Una pasada de coordenadas	$\leq 4\,000$

Incluso su variante más barata supera con holgura a $B = 14\,400$. Además, una configuración al azar suele quedar por debajo del 5 % y el puntaje es todo-o-nada, así que con $n_0 = 2$ casi todas marcarían 0 % y no se podrían separar.

6. Proceso de optimización y resultados

Esta sección describe la búsqueda real que produjo `entrega.json`, con el modelo `qwen17b`. Cada resultado remite a la celda del notebook de búsqueda (`notebooks/busqueda.ipynb`, secciones B1 a B7) o a un archivo de `resultados/`. Lo comprobado y lo que es hipótesis se distinguen en cada caso. En todo el texto, *mejor encontrada* no significa óptima.

6.1. Método, parámetros y parada

Se usó una sola técnica, la búsqueda por coordenadas de la sección 4 (D3), con dos arranques de puntos de partida distintos. No se compararon otros métodos con el modelo real.

Parámetro	Valor
Modelo, temperatura, semilla	`qwen17b`, 0,0, semilla 1 (D2)
Espacio	5 ranuras de 8 opciones, $\|\mathcal{C}\| = 32\,768$
Lote de búsqueda	100 instancias de `datos_visibles.json`
Presupuesto	$B = 14\,400$ generaciones nuevas (D11, D17)
Tope de pasadas	$P = 3$ en los dos arranques (D18)
Orden de ranuras	Arranque 0: rol, estrategia, formato, estilo, cierre. Arranque 1: formato, rol, estilo, estrategia, cierre
Criterio de aceptación	Mejora estricta de la precisión; en empate se conserva la actual
Caché	Se guarda en Drive; no se cobra una (configuración, instancia, semilla) ya generada

La decisión D17 fijaba hasta dos pasadas por arranque. El arranque 0 usó una tercera como verificación y quedó dentro de B; por eso el tope quedó en tres (D18). La decisión se aprobó el 5 de noviembre de 2026, después de ejecutar los arranques, así que regulariza lo que se hizo; no cambió ninguna corrida.

Condición de parada (celda B4). Un arranque termina por la primera de estas causas: una pasada completa sin cambios aceptados (convergencia, óptimo local por ranura, teorema 4); llegar a P pasadas; o agotar B, caso en que se devuelve la mejor configuración vista. Los dos arranques pararon por convergencia.

Reproducibilidad. El punto de partida de cada arranque lo fija una semilla. Cada paso se escribe en un CSV (`resultados/busqueda/historial_arranque_*.csv`) con la configuración, su precisión, la mejor hasta ahí, las generaciones acumuladas y la decisión (acepta o descarta). La celda B6 recalcula desde esos CSV los conteos y falla si no coinciden con los registrados.

6.2. Registro de las evaluaciones

	Arranque 0	Arranque 1
Punto de partida (rol, estr., formato, estilo, cierre)	3, 0, 6, 7, 2	6, 7, 0, 4, 3
Precisión de partida	3 %	9 %
Configuración final	1, 1, 3, 3, 3	1, 7, 3, 5, 3
Precisión final (búsqueda)	20 %	17 %
Pasadas	3 (la tercera sin cambios)	2 (la segunda sin cambios)
Configuraciones distintas evaluadas	92	28
Generaciones (distintas × 100)	9 200	2 800
Cambios aceptados	8	4

Los cambios aceptados fueron, en orden, con la precisión que alcanzó cada uno:

- Arranque 0: rol 0 (6 %), rol 1 (8 %), estrategia 1 (9 %), estrategia 4 (10 %), formato 3 (11 %), cierre 3 (15 %), estrategia 1 (19 %) y estilo 3 (20 %). La estrategia pasó de 1 (paso 10) a 4 (paso 13) en la primera pasada y volvió a 1 en el paso 47, de la segunda, cuando ya habían cambiado formato y cierre: la mejor opción de una ranura depende de lo que valen las demás.
- Arranque 1: formato 3 (12 %), rol 1 (13 %), estilo 0 (15 %) y estilo 5 (17 %).

Las ocho mejores configuraciones de la unión de los dos arranques, por precisión sobre el lote de búsqueda:

Precisión	rol	estrategia	formato	estilo	cierre
20 %	1	1	3	3	3
19 %	1	1	3	7	3
19 %	1	1	6	7	3
17 %	1	1	3	0	3
17 %	1	1	3	5	3
17 %	1	4	3	3	3
17 %	1	7	3	5	3
16 %	1	2	3	7	3

Las ocho comparten rol 1 y cierre 3, y siete de ocho comparten formato 3. Con 100 instancias, la desviación típica de una proporción cercana a 0,15 es de unos 3,6 puntos (raíz de $0{,}15 \cdot 0{,}85/100$). Es una estimación teórica, no medida entre semillas: diferencias de uno a tres puntos no distinguen configuraciones.

Presupuesto gastado. Como el caché no cobra lo ya generado, el gasto real es el de la unión de configuraciones, no la suma de los dos arranques: 117 configuraciones, 11 700 generaciones, el 81,25 % de B. Los dos arranques compartieron tres configuraciones, de modo que la suma por

sesión es 12 000. Que el caché evitara repetirlas es una inferencia, porque el CSV no registra aciertos de caché. Aparte quedan 20 generaciones de pruebas del caché y 11 de un diagnóstico hecho en otra cuenta; si se cuentan, el total es 11 731 (81,5 %). La validación (87) no entra en B. En cualquier lectura el gasto queda por debajo de 14 400.

6.3. Trazas de fallos y decisiones derivadas

Tras el arranque 0 se analizó en qué instancias fallaba su mejor configuración (80 de 100), por familia de restricción y por la primera restricción violada, y con eso se recortó el espacio del arranque 1 (celdas B5; `resultados/busqueda/fallos_*.txt` y `analisis_busqueda.md`). Con la mejor encontrada de cada arranque, 27 familias fallan siempre y 23 de ellas son comunes; las primeras restricciones violadas más frecuentes también coinciden (`last_word:last_word_sent`, `copy:copy` y `new:copy_span_idx`).

Ranura	Excluidas en el arranque 1	Fundamento	Estado
rol	0, 3, 4, 7	Fijan una primera línea literal (leído del catálogo); se supuso que chocan con `first_word`	La primera línea literal está comprobada. El choque es hipótesis: con el caché, esas opciones aciertan 0 o 1 de 11 instancias de `first_word_answer`, frente a 3 de 11 con rol 1; es una tendencia con 11 casos
formato	1, 4, 5	Nunca pasaron de 10 % en el arranque 0	Contrastado con el historial
estrategia	3	Nunca pasó de 10 % en las pasadas 1 y 2	**No se sostuvo**: en la pasada 3 llegó a 15 %
cierre	todas menos la 3	La 3 es la única que no agrega texto tras la respuesta	Comprobado con el catálogo. Que por eso supere a las demás (20 % frente a 13 % como máximo) es hipótesis

Con esas exclusiones el arranque 1 buscó en $4 \cdot 5 \cdot 7 \cdot 8 \cdot 1 = 1\,120$ de las 32 768 configuraciones (3,4 %). En el contexto final del arranque 0, ninguna opción excluida superó el 15 % frente al 20 %, así que las exclusiones no habrían cambiado su mejor; eso se midió solo en ese contexto.

Vecindad completa del arranque 1 (experimento posterior). La salida y el código están en `resultados/busqueda/vecindad_arranque_1.txt`. Tras congelar la configuración se evaluaron los 35 vecinos por una ranura de la mejor del arranque 1 (rol 1, estrategia 7, formato 3, estilo 5, cierre 3, 17 %), sin las exclusiones. Los 20 ya evaluados salieron del caché; los 15 restantes costaron a lo sumo 1 500 generaciones, hechas en otra cuenta de Colab con una copia del caché y fuera del conteo de 11 700 de la búsqueda (con ellas el total no pasaría de 13 200, dentro de B). Resultado: ningún vecino supera al 17 % y el máximo es 17 %, un empate con la estrategia 1, que ya estaba en el historial. Entre las 15 opciones antes excluidas el máximo fue 14 % (rol 3), con rol 0, 4 y 7 entre 10 % y 13 %, formato 1, 4 y 5 entre 8 % y 9 %, estrategia 3 en 12 % y cierre entre 8 % y 13 %. Por tanto, el 17 % del arranque 1 es un óptimo local por ranura completo y no solo respecto del subespacio permitido, y las exclusiones no ocultaban ninguna mejora en ese punto. Se confirma que la premisa de la estrategia 3 era falsa, aunque no cambió el resultado. Esto no modifica la configuración entregada (20 %), y no es evidencia independiente de que ninguna de las dos sea un óptimo global.

Qué se descartó y qué cambió. Las opciones de la tabla se descartaron por esta evidencia, y el arranque 1 se dedicó a la región restante. El resultado fue distinto: una interacción entre estrategia y estilo. Con rol 1, formato 3 y cierre 3, la estrategia 1 da 20 % con el estilo 3 y 17 % con el 5, y la estrategia 7 da 11 % con el estilo 3 y 17 % con el 5. Esa dependencia ya anticipaba un óptimo local según el punto de partida; es lo que ilustran los dos arranques.

Lo que las trazas no permiten concluir. La frase *"Before each part of the answer..."* pertenece al texto de la estrategia 7 (comprobado en el catálogo) y aparece en los primeros 300 caracteres de 32 de los 83 fallos del arranque 1 y en 0 de los 80 del arranque 0. No se sabe cuántas de las 17 salidas acertadas la contienen, así que que cause los fallos es una hipótesis. Tampoco hay datos por familia de las otras 115 configuraciones evaluadas: ninguna fila del análisis demuestra que una opción cause un fallo. Las familias de copia (`copy:copy`, `new:copy_span_idx`) acertaron 0 de 11 con cuatro vecinos y 1 de 11 con una configuración sin añadidos; la hipótesis de que el texto de las opciones, que forma parte del prompt, se copie junto con el enunciado no está comprobada.

6.4. Validación final

La configuración se congeló el 1 de noviembre de 2026 por una regla fijada antes de validar: la de mayor precisión en búsqueda entre las mejores de cada arranque (20 % frente a 17 %). Esa diferencia de tres puntos está dentro del ruido; la elección sale de la regla, no de una diferencia demostrada. Quedó con el registro y las huellas de los archivos fuente en `docs/bitacora/2026-10.md` y con la etiqueta `config-congelada`.

Después se invocó `oraculo.validar(config, n=None)` **una sola vez** (celda B7; copia ejecutada en `resultados/validacion/busqueda_validado_2026-11-01.ipynb`). El resultado fue:

Configuración	rol 1, estrategia 1, formato 3, estilo 3, cierre 3, temperatura 0,0
Precisión de búsqueda	20,0 % sobre 100 instancias
Precisión de validación	18,4 % sobre 87 instancias (16 aciertos)

Los 16 aciertos son un cálculo propio ($0{,}184 \cdot 87$); la salida del notebook solo imprime el porcentaje. Las familias con más fallos, por primera restricción violada, fueron `last_word_sent` (6), `punctuation_dot` (6), `repeat_phrase` (5), `repeat_prompt` (5), `no_adjacent_consecutive` (4), `letter_counting` (4) y `last_word_answer` (4).

Interpretación. La diferencia entre búsqueda y validación es de 1,6 puntos, dentro del ruido. Es coherente con que el 20 % fuera algo optimista, porque es el máximo entre 117 configuraciones medidas sobre las mismas instancias, pero no lo prueba. Para una configuración fijada antes de mirar los datos, la desigualdad de Hoeffding (sección 2) da con 87 instancias un error de hasta 0,146 con $\delta = 0{,}05$; la cifra de 18,4 % es, por tanto, una estimación gruesa, compatible con valores entre unos 4 % y 33 %. Esa cota supone instancias independientes.

Qué comprueba y qué no. La validación comprueba que la configuración elegida no tiene un desempeño muy distinto en instancias que no intervinieron en la búsqueda. No es la evaluación final: el test de la docente es privado y puede traer familias de restricciones ausentes de los datos visibles, que quedan fuera del alcance del equipo. La validación no se repite ni se usa para elegir (D12).

6.5. Límites

- La configuración entregada es la mejor encontrada, un óptimo local por ranura. No se afirma que sea un óptimo global ni de un subcatálogo.

- El arranque 1 no es independiente del 0: usó exclusiones derivadas de sus trazas. Su coincidencia en rol 1, formato 3 y cierre 3 no es evidencia independiente.
- Durante la búsqueda, el 17 % del arranque 1 solo era óptimo local respecto del subespacio permitido: de sus 35 vecinos por una ranura se evaluaron 20 y 15 quedaron sin evaluar por las exclusiones. Esos 15 se evaluaron después, como experimento aparte (véase arriba); no intervinieron en la elección de la configuración entregada.
- La premisa de excluir la estrategia 3 resultó falsa: llegó a 15 % en la pasada 3 del arranque 0 y a 12 % como vecino del arranque 1. Eso no cambia la configuración ganadora.
- Solo hay dos arranques, y no se midió la variabilidad entre semillas.
- La velocidad (1,99 s por generación) se midió con una sola configuración; otras podrían tardar más.
- Los barridos de $T(n)$ se hicieron solo para el exacto y con el modelo sintético (sección 3). No se hizo el exacto sobre un subcatálogo con el modelo real (D17), y la cota de costo de la sección 4 se contrasta solo con las generaciones observadas, no con un barrido de tiempos.
- Con las 100 instancias de búsqueda y las 87 de validación, los resultados de la sección no generalizan sin supuestos a otras familias de restricciones.

6.6. Dónde está cada cosa

Elemento	Ubicación en el repositorio
Configuración entregada	`entrega.json` (grupo ADAG1)
Notebook de búsqueda	`notebooks/busqueda.ipynb`, celdas B1 a B7
Notebook de evaluación	`notebooks/evaluacion.ipynb`
Historiales y resúmenes	`resultados/busqueda/`
Análisis de fallos	`resultados/busqueda/analisis_busqueda.md`
Copia de la validación	`resultados/validacion/`
Registro del congelamiento	`docs/bitacora/2026-10.md`

7. Discrepancias y limitaciones

7.1. Discrepancias entre las fuentes

El repositorio base manda en datos y ejecución; el enunciado, en entregables, pesos y rigor (D1). La guía de la primera entrega manda sobre el enunciado en esta entrega y en la reducción (D13). La tabla registra las diferencias y la decisión tomada en cada caso.

Tema	Enunciado	Repositorio base	Decisión
$\lvert\mathcal{C}\rvert$	3840 y 144; con temperatura, 432	32 768; con temperatura, 98 304	32 768 (D2)
Opciones por categoría	$k_{\text{máx}} = 4$, con máximo $4^5 = 1024$	8 por ranura	8 (D2)
Instancias	$n = 150$	100 de búsqueda y 87 de validación	$n = 100$ (D2)
Categorías	Rol, Estrategia, Formato, Verificación, Recordatorio	`rol`, `estrategia`, `formato`, `estilo`, `cierre`	Las del repositorio (D1)
Instancias sintéticas	Todas sintéticas	Modelo real en Colab	Ambas, con funciones distintas (D7)
Exacto	Fuerza bruta o programación dinámica en el cronograma; backtracking en §4.3	Sin definir	Backtracking, con fuerza bruta como referencia (D3)
Reducción	Cobertura Máxima	La guía indica 3-SAT	3-SAT (D9, D13)

Otras inconsistencias comprobadas:

- Según `datos_visibles.json`, hay 100 instancias de búsqueda, 87 de validación y 263 descartadas, aunque el cuaderno del repositorio base menciona 150 y 300. Se usan los valores del archivo.
- La fórmula de Successive Halving solo es exacta si $\lvert\mathcal{C}\rvert$ es potencia de 2.
- Con 5 categorías fijas, $\lvert\mathcal{C}\rvert$ es constante y el problema es polinomial. Por eso L y los k_i son parte de la entrada de la reducción (D9).
- La fórmula de coordenadas omite L porque lo toma constante; con L variable, $T = O(I \cdot L \cdot k_{\text{máx}} \cdot n)$.
- D17 fijó hasta 2 pasadas por arranque, pero el arranque 0 usó una tercera como verificación. El gasto total, 11 700 generaciones de la unión, quedó dentro de B.

7.2. Limitaciones de los resultados

- **Alcance de la configuración elegida.** Es un óptimo local por ranura: se evaluaron sus 35 vecinos que difieren en una sola ranura y ninguno la supera. No se afirma que sea un óptimo global ni el óptimo de un subcatálogo.
- **Sesgo de selección.** El 20 % es el máximo entre 117 configuraciones medidas sobre las mismas 100 instancias, así que probablemente es optimista. La cifra sin ese sesgo es la de validación (Sección 6).
- **Ruido.** La diferencia con el arranque 1 (3 puntos) está dentro de un ruido estimado en unos 4 puntos con 100 instancias. Esa estimación es teórica, no una medición entre semillas, y la elección sale de una regla fijada antes de validar.
- **Arranques no independientes.** El arranque 1 usó exclusiones derivadas del arranque 0, de modo que no mide el riesgo de óptimos locales como lo haría una repetición independiente. En su archivo de historial, la columna `arranque` dice 0.
- **Mejor configuración del arranque 1.** Su 17 % no es un óptimo local completo: 15 de sus 35 vecinos no se evaluaron por esas exclusiones.
- **Una premisa falsa.** Una de las exclusiones suponía que la opción 3 de `estrategia` nunca pasaba de 10 %, pero alcanzó 15 % en la tercera pasada del arranque 0. Esto no cambia la configuración elegida.

- **Vínculos entre opciones y fallos.** Siguen siendo hipótesis, no relaciones causales demostradas.
- **Datos privados.** Pueden traer familias de restricciones que no están en los datos visibles, de modo que la precisión de esta entrega no anticipa la de ese conjunto.

Referencias

[1] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2-3):235–256, 2002.

[2] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.

[3] Richard M. Karp. Reducibility among combinatorial problems. In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972.

Anexo: decisiones citadas

El texto cita entre paréntesis las decisiones del proyecto por su identificador (D1, D2, ...). Se tomaron antes de ejecutar y no cambian sin evidencia nueva. La tabla resume las citadas; la lista completa, con su justificación, está en el repositorio (`AGENTS.md`, sección 5).

ID	Decisión
D1	El repositorio base manda en datos y ejecución; el enunciado, en entregables, pesos y rigor. Las diferencias se registran en la sección 7.
D2	$L = 5$, $k_i = 8$, $\lvert\mathcal{C}\rvert = 32\,768$, $n = 100$, validación con 87 instancias, temperatura 0,0, modelo `qwen17b` y semilla 1.
D3	Técnicas elegidas: backtracking, búsqueda por coordenadas y reparto uniforme frente a UCB1.
D4	Successive Halving no se implementa: su costo supera el presupuesto y con pocas instancias iniciales casi todas las configuraciones darían 0 %.
D5	Cota del exacto: aciertos más instancias restantes; si no alcanza a la mejor, se abandona la configuración.
D7	El exacto se mide con el modelo sintético; el modelo real da los resultados de precisión.
D9	Reducción desde 3-SAT en su versión de decisión, con L y k_i como parte de la entrada.
D11	B cuenta las generaciones nuevas por (configuración, instancia, semilla).
D12	La validación con 87 instancias se usa una sola vez, sobre la configuración ya elegida.
D13	La guía de la primera entrega manda sobre el enunciado en esta entrega y en la reducción.
D17	Búsqueda real con $B = 14\,400$ generaciones y dos arranques.
D18	Tope de tres pasadas por arranque, el que se usó en los dos arranques (D17 preveía dos). Se aprobó después de ejecutarlos.